Saturacion QA — Analisis Completo, Estrategias y Plan de Implementacion

Proyecto: Intrale Platform | **Fecha:** 2026-04-06 | **Autor:** Claude Code (Opus 4.6)

Indice

1. Diagnostico del Sistema
2. Causas Raiz de la Saturacion
3. Estrategias Evaluadas
4. Decisiones Tomadas
5. Arquitectura Final: Build → QA con Backend Remoto
6. Deploy del Backend a Lambda (detalle)
7. Restriccion de Paralelismo de QA
8. Consumo de RAM Comparativo
9. Prerequisitos de Implementacion
10. Riesgos y Mitigaciones
11. Plan de Implementacion por Fases
12. Resumen Ejecutivo

1. Diagnostico del Sistema

1.1 Hardware

Recurso	Especificacion	Evaluacion
CPU	Intel Core i5-1145G7 — 4 cores / 8 threads @ 2.6 GHz	Limitado
RAM	16 GB (15.7 GB visible)	Ajustado
Disco	235.5 GB total — 24.4 GB libres (10.4%)	Ajustado
GPU	Intel Iris Xe (integrada)	Sin dedicada

1.2 Consumo de Memoria Baseline (sin QA)

Componente	Memoria	Notas
Java (procesos Claude/builds)	**~5,500 MB**	2 procesos principales + residuales
Chrome	~1,200 MB	Multiples tabs
Node.js (pipeline, hooks)	~500 MB	Dashboard, commander, hooks
Windows Defender (MsMpEng)	~486 MB	Siempre activo
Edge WebView2 (WhatsApp, etc)	~600 MB	Apps de escritorio
Sistema (Explorer, Terminal, otros)	~800 MB	Windows base
Total en uso	**~14.9 GB de 16 GB — Solo 1.5 GB libres**	

1.3 Demanda del QA Stack Actual (todo local)

Componente	RAM	CPU Cores
Docker Desktop + DynamoDB Local + Moto/Cognito	~1,300 MB	~1
Backend Ktor (`gradlew :users:run`)	~1,200 MB	1-2
Emulador Android (1 shard, 1536 MB)	~2,000 MB	2
Build APK (`assembleClientDebug`)	~2,500 MB	2-4
Maestro + ADB	~400 MB	0.5
Total QA	**~7,400 MB**	**6-9 cores**

DEFICIT: El sistema tiene 1.5 GB libres pero QA necesita ~7.4 GB adicionales. Deficit de ~6 GB que fuerza swap masivo, degradando el sistema a un estado impredecible.

2. Causas Raiz de la Saturacion

#	Causa	Severidad	Detalle
1	**RAM insuficiente para carga concurrente**	Critica	16 GB no alcanza para agentes (~5.5 GB) + QA (~7 GB) + sistema (~3 GB) = 15.5 GB sin margen. Un build activo (issue #2017) lo lleva a ~17+ GB.
2	**Procesos Java residuales**	Alta	Gradle daemons y builds previos quedan como huerfanos consumiendo hasta 4 GB. Aunque `org.gradle.daemon=false` , otros procesos (backend, compilaciones) persisten.
3	**CPU limitada (4 cores reales)**	Alta	Emulador (2 cores) + Gradle (2-4 cores) + agentes = overcommit severo. Con 3 shards (6 cores) es fisicamente imposible.
4	**Sin orquestacion de ciclo de vida**	Media	No hay mecanismo para suspender agentes → liberar RAM → ejecutar QA → reanudar. Cada componente se levanta independientemente.
5	**QA compila redundantemente**	Media	La fase QA recompila el APK y/o el backend aunque la fase Build previa ya los genero. Picos de 2.5 GB innecesarios.
6	**Stack local pesado (Docker + Backend + DB)**	Media	DynamoDB local + Moto + Backend Ktor consumen ~2.5 GB cuando ya existe infraestructura remota equivalente en AWS.

3. Estrategias Evaluadas

3.1 Mapa completo

Estrategia	RAM liberada	Costo	Esfuerzo	Decision
A. Backend remoto (Lambda + API GW)	~2.5 GB	$0	Simple	ADOPTADA
B. Liberacion pre-QA orquestada	~3-5 GB	$0	Simple	ADOPTADA
C. APK de la fase Build (sin recompilar)	~2.5 GB	$0	Simple	ADOPTADA
D. Reducir footprint del emulador	~500 MB	$0	Simple	DESCARTADA
E. Upgrade RAM a 32 GB	Total	~$60	Simple	FUTURO
F. Modo QA exclusivo (matar todo)	~8-10 GB	$0	Simple	FALLBACK
G. Maestro Cloud	~2.2 GB	$10-40/mes	Medio	FUTURO
H. QA E2E en GitHub Actions	Total	Variable	Grande	FUTURO
I. DynamoDB remoto + backend local	~800 MB	$0	Medio	DESCARTADA
J. Staging environment dedicado en AWS	~2.5 GB	$0 (Free Tier)	Medio	FUTURO

3.2 Estrategias adoptadas (detalle)

A. Backend Remoto — Emulador pega a Lambda en AWS

Eliminar Docker Desktop, DynamoDB local, Moto/Cognito y el backend Ktor de la maquina. El emulador apunta directamente al API Gateway que ya existe en AWS.

Infraestructura remota existente:

Servicio	Endpoint	Estado
API Gateway	`https://mgnr0htbvd.execute-api.us-east-2.amazonaws.com/dev/`	Activo
Lambda (kotlinTest)	Detras de API Gateway	Activo
DynamoDB	us-east-2 (business, users, userbusinessprofile)	Activo
Cognito	User Pool en us-east-2	Activo

PRO: -2.5 GB RAM; elimina Docker; la app no esta productiva, sin riesgo; usa servicios reales (mas confiable que mocks)

CON: Requiere internet; deploy previo del JAR a Lambda; latencia de red (~100-200ms vs ~5ms, irrelevante para Maestro)

B. Liberacion Pre-QA Orquestada

Antes de iniciar QA: activar QA Priority Window (bloquea nuevos agentes), esperar que builds terminen, matar Java residuales, verificar RAM libre >= 4 GB.

PRO: +3-5 GB RAM; automatizable; sin costo; compatible con cualquier estrategia

CON: Introduce delay (espera de builds); interrumpe trabajo temporalmente

C. APK de la Fase Build (sin recompilar en QA)

La fase Build (agente `/builder`) ya compila el APK. QA reutiliza ese artefacto directamente en vez de lanzar otro Gradle build de ~2.5 GB.

PRO: Elimina el pico mas grande de consumo; QA arranca inmediatamente sin compilar

CON: Requiere que Build genere los artefactos en ubicacion conocida

3.3 Estrategias descartadas (con justificacion)

D. Reducir footprint del emulador — DESCARTADA

Bajar RAM del emulador de 1536 MB a 1024 MB y cores de 2 a 1.

Razon de descarte: Afecta la fluidez del video (screenrecord) y la estabilidad de los tests Maestro. El ahorro de ~500 MB no justifica la degradacion de calidad de evidencia QA. El emulador se mantiene en **1536 MB / 2 cores**.

I. DynamoDB remoto + backend local — DESCARTADA

Mantener backend Ktor local pero apuntando a DynamoDB/Cognito remotos.

Razon de descarte: Si ya vamos a usar AWS remoto, no tiene sentido mantener el backend local (~1.2 GB). Mejor usar Lambda directamente (Estrategia A). Esta opcion solo tendria sentido si el issue modifica codigo de backend y necesitas testearlo antes de deploy — en ese caso el agente de dev ya tiene el backend corriendo en su worktree.

4. Decisiones Tomadas

Decisiones confirmadas

Punto	Decision	Razon
Emulador Android	**Mantener 1536 MB / 2 cores**	Reducir afecta calidad de video y estabilidad de Maestro
Compilacion del APK	**Usar APK de la fase Build previa**	Build ya compila antes de QA; evita pico de 2.5 GB
Backend para QA	**Lambda remoto en AWS**	Elimina Docker + Ktor local; la app no es productiva
Deploy del backend	**Via GitHub Actions (`gh workflow run`)**	Sin AWS CLI local; secrets seguros; 0 RAM local
Paralelismo QA	**Secuencial (1 issue a la vez)**	Backend remoto compartido; pipeline ya encola secuencialmente

5. Arquitectura Final: Build → QA con Backend Remoto

5.1 Diagrama de flujo entre fases

```
          FASE BUILD                              FASE QA
  (agente builder, ~2.5 GB)                (agente qa, ~2 GB)
┌──────────────────────────┐        ┌──────────────────────────────┐
│                          │        │                              │
│  1. shadowJar            │        │  1. Liberacion pre-QA        │
│     → users-all.jar      │──┐     │     → Activar QA Priority Window │
│                          │  │     │     → Kill Java residuales   │
│  2. assembleClientDebug  │  │     │     → Verificar RAM >= 4 GB  │
│     → app-client-debug.apk │──┤   │                              │
│     (SIN LOCAL_BASE_URL) │  │     │  2. Deploy backend a Lambda  │
│                          │  │     │     → gh workflow run main.yml │
│  3. Copiar artefactos a  │  │     │     → Esperar completion     │
│     qa/artifacts/        │  │     │     → Health-check remoto    │
│                          │  │     │                              │
└──────────────────────────┘  │     │  3. Instalar APK en emulador │
                              ├────→│     → adb install (de artifacts/)│
       Artefactos:            │     │                              │
       • users-all.jar        │     │  4. Ejecutar Maestro + video │
       • composeApp-client-   │     │     → Tests E2E contra Lambda │
         debug.apk            │     │     → screenrecord 1536MB/2cores │
       • BUILD_COMMIT         │     │                              │
                              │     │  5. Recolectar evidencia     │
                              │     │     → Videos, logs, reportes │
                              │     │     → Desactivar Priority Window │
                              │     └──────────────────────────────┘
                              │
                              │    Emulador apunta a:
                              │    https://mgnr0htbvd.execute-api.
                              │    us-east-2.amazonaws.com/dev/
                              │    (default de BuildKonfig)
```

5.2 Lo que genera la fase Build

FASE BUILD — Genera ambos artefactos

Artefacto	Comando	Output	Destino
Backend JAR	`./gradlew :users:shadowJar --no-daemon`	`users/build/libs/users-all.jar`	QA deploy a Lambda
APK Client	`./gradlew :app:composeApp:assembleClientDebug --no-daemon`	`app/composeApp/build/outputs/apk/client/debug/`	QA instalar en emulador

> **Clave:** El APK se compila **sin** `-PLOCAL_BASE_URL`. Asi usa el default de BuildKonfig: `https://mgnr0htbvd.execute-api.us-east-2.amazonaws.com/dev/`

Al finalizar, Build copia artefactos a ubicacion fija:

```
mkdir -p qa/artifacts/
cp users/build/libs/users-all.jar qa/artifacts/
cp app/composeApp/build/outputs/apk/client/debug/composeApp-client-debug.apk qa/artifacts/
echo "$(git rev-parse HEAD)" > qa/artifacts/BUILD_COMMIT
echo "$(date -u +%Y%m%d-%H%M%S)" > qa/artifacts/BUILD_TIMESTAMP
```

5.3 Lo que hace la fase QA

FASE QA — Solo emulador + Maestro (sin compilar, sin Docker, sin backend local)

```
PASO 1: LIBERACION PRE-QA (~30s)
├── Activar QA Priority Window en pulpo.js (bloquea nuevos agentes)
├── Esperar que builds activos terminen (max 10 min)
├── Matar procesos Java/Node residuales
├── Verificar RAM libre >= 4 GB
└── Si insuficiente → abortar con error y notificar

PASO 2: DEPLOY BACKEND REMOTO (~3-5 min)                          ← REMOTO
├── git push origin [rama-del-issue]
├── gh workflow run main.yml --ref [rama-del-issue]
├── Polling: gh run list --workflow=main.yml --limit=1
│   └── Cada 30s hasta status=completed
├── Verificar conclusion=success
├── Health-check: POST /intrale/signin → esperar HTTP 400
│   └── 400 = Lambda viva y respondiendo (Bad Request esperado)
└── Si falla → abortar + notificar

PASO 3: PREPARAR EMULADOR (~40s con snapshot)                      ← LOCAL
├── Verificar/arrancar virtualAndroid
│   └── -snapshot qa-ready -no-audio -no-boot-anim -no-window
│   └── -memory 1536 -cores 2 -gpu auto
├── Esperar sys.boot_completed=1 (timeout: 180s)
├── Desactivar animaciones
└── Esperar 15s de estabilizacion

PASO 4: INSTALAR APK (~15s)                                        ← LOCAL
├── Localizar APK: qa/artifacts/composeApp-client-debug.apk
├── Verificar BUILD_COMMIT coincide con el commit bajo test
├── adb install -r [APK]
└── Verificar instalacion exitosa

PASO 5: EJECUTAR TESTS MAESTRO + VIDEO (~2-5 min)                  ← LOCAL
├── Iniciar screenrecord (720x1280 @ 2Mbps)
├── maestro test .maestro/flows/
├── Detener screenrecord
├── Extraer video del emulador
```

```
└── Generar reporte de resultados

PASO 6: RECOLECTAR EVIDENCIA + CLEANUP (~30s)                    ← LOCAL
├── Copiar videos a qa/evidence/
├── Generar summary.md
├── Subir evidencia (screenshots, video, logs)
└── Desactivar QA Priority Window → pipeline reanuda
```

6. Deploy del Backend a Lambda (detalle)

6.1 Estado actual de la infraestructura

Componente	Estado	Detalle
AWS CLI local	No instalado	No hay `aws` en PATH
Credenciales AWS locales	No configuradas	No hay `~/.aws/credentials`
Lambda `kotlinTest`	Activo	Region us-east-2, detras de API Gateway
GitHub Actions deploy	Funcional	Secrets configurados: AWS_DEPLOYER_SECRET_ID/KEY
`gh` CLI	Instalado	v2.86.0, autenticado

6.2 Opciones de deploy evaluadas

Opcion A — Deploy via GitHub Actions (ELEGIDA)

Concepto: El agente QA dispara el workflow de deploy existente usando `gh` CLI. GitHub Actions compila el JAR con secrets inyectados y lo deploya a Lambda. Cero consumo de RAM local.

Flujo paso a paso:

```
# 1. Push de la rama del issue a origin
git push origin agent/2017-fix-saturacion

# 2. Disparar el workflow de deploy apuntando a la rama
gh workflow run main.yml --ref agent/2017-fix-saturacion

# 3. Esperar a que termine (~3-5 min)
while true; do
  STATUS=$(gh run list --workflow=main.yml --limit=1 --json status,conclusion -q '.[0].status')
  if [ "$STATUS" = "completed" ]; then break; fi
  sleep 30
done

# 4. Verificar conclusion
CONCLUSION=$(gh run list --workflow=main.yml --limit=1 --json conclusion -q '.[0].conclusion')
if [ "$CONCLUSION" != "success" ]; then
  echo "ERROR: Deploy fallo"; exit 1
fi

# 5. Health-check: Lambda actualizada y respondiendo
HTTP_CODE=$(curl -s -o /dev/null -w "%{http_code}" \
  -X POST "https://mgnr0htbvd.execute-api.us-east-2.amazonaws.com/dev/intrale/signin" \
  -H "Content-Type: application/json" \
  -d '{"email":"test","password":"test"}')
# Esperamos 400 (Bad Request = Lambda viva, validando input)
if [ "$HTTP_CODE" != "400" ]; then
  echo "WARN: Health-check retorno $HTTP_CODE (esperado 400)"
fi
```

Que pasa internamente en GitHub Actions:

```
GitHub Actions Runner (ubuntu-latest, 7 GB RAM):
|
```

```
├── 1. Checkout del codigo (rama del issue)
├── 2. Setup JDK 21 (Temurin) + Gradle
├── 3. Inyectar secrets en application.conf:
│      sed "s/REGION_VALUE/${REGION}/gi"
│      sed "s/ACCESS_KEY_ID/${ACCESS_KEY_ID}/gi"
│      sed "s/SECRET_ACCESS_KEY/${ESCAPED_SECRET}/gi"
│      sed "s/USER_POOL_ID/${USER_POOL_ID}/gi"
│      sed "s/CLIENT_ID/${CLIENT_ID}/gi"
├── 4. ./gradlew build → genera users-all.jar (shadow JAR ~50-80 MB)
├── 5. yvesgurcan/deploy-lambda-function:
│      → aws lambda update-function-code
│        --function-name kotlinTest
│        --zip-file fileb://users/build/libs/users-all.jar
│        --region (AWS_DEPLOY_REGION)
│      → Credenciales: AWS_DEPLOYER_SECRET_ID / AWS_DEPLOYER_SECRET_KEY
└── 6. Lambda kotlinTest actualizada con el codigo de la rama
```

Cambio requerido en workflow:

```
# .github/workflows/main.yml — Agregar workflow_dispatch:
on:
  push:
    branches: [ main ]
    paths: [...]
  repository_dispatch:
  workflow_dispatch:          # ← NUEVO: permite disparo desde gh CLI
    inputs:
      deploy_only:            # ← OPCIONAL: para future uso
        type: boolean
        default: false
```

PRO: Sin AWS CLI local; sin secrets en la maquina; 0 MB de RAM; reutiliza pipeline probado; el JAR se compila en runner con 7 GB RAM

CON: Esperar ~3-5 min al workflow; requiere agregar `workflow_dispatch` al trigger

Opcion B — Deploy local via AWS CLI — DESCARTADA

Instalar AWS CLI + configurar credenciales + compilar shadowJar localmente + `aws lambda update-function-code`.

Razon de descarte:

- Requiere almacenar secrets AWS en disco local (seguridad reducida)
- El shadowJar consume ~2 GB de RAM para compilar (justamente lo que queremos evitar)
- Requiere inyectar secrets en `application.conf` manualmente (sed + archivo de secrets)
- Setup adicional: instalar AWS CLI, configurar profile `intrale-deployer`

Opcion C — Hibrido: Build en GitHub + Deploy local — DESCARTADA

GitHub Actions compila el JAR y lo sube como artefacto. QA local lo descarga y lo deploya con AWS CLI.

Razon de descarte: Aun requiere AWS CLI + credenciales locales para el paso de deploy. Si ya se usa GitHub Actions para compilar, mejor que tambien deploya (Opcion A).

6.3 Comparativa final de opciones de deploy

Criterio	A. GitHub Actions ✓	B. AWS CLI local ✗	C. Hibrido ✗
Setup local	Nada	AWS CLI + secrets	AWS CLI + secrets
RAM local consumida	0 MB	~2 GB (shadowJar)	0 MB
Tiempo de deploy	3-5 min	2-3 min	4-6 min
Seguridad de secrets	Solo en GitHub	En disco local	En disco local
Complejidad	Baja (1 cambio)	Media (4 pasos)	Media (3 pasos)

7. Restriccion de Paralelismo de QA

> **Restriccion:** Con un solo Lambda remoto compartido (`kotlinTest`), no se pueden ejecutar dos QA E2E de issues distintos simultaneamente. Los datos de test (usuarios, negocios, estados) se pisarian entre si.

7.1 Mitigacion actual (ya implementada)

- El pipeline (`pulpo.js`) ya limita QA a **1 agente secuencial**
- `config.yaml` : QA usa 1 shard, si 2 agentes llegan juntos uno espera en cola
- La QA Priority Window garantiza que el agente encolado tiene prioridad sobre nuevos dev
- **En la practica no cambia nada** — QA ya era secuencial

7.2 Evolucion futura (si se necesita paralelismo)

Opcion	Descripcion	Complejidad	Cuando
Namespacing por issue	Cada QA usa datos con prefijo: `test-2017-user@intrale.com`	Medio	Si el equipo crece a 3+ devs
Lambda staging fija	Crear `kotlinStaging` permanente con tablas separadas	Medio	Cuando la app sea productiva
Lambdas efimeras	Crear Lambda + tablas temporales por issue, destruir al finalizar	Grande	Solo si volumen de QA es alto

Recomendacion: No es necesario ahora. Con el modelo Kanban de 1 QA a la vez y la app no productiva, el backend remoto compartido alcanza perfecto.

8. Consumo de RAM Comparativo

8.1 Antes vs. despues (durante fase QA)

Componente	Antes (local)	Despues (remoto)	Ahorro
Docker Desktop	~500 MB	**0 MB**	-500 MB
DynamoDB Local	~500 MB	**0 MB** (AWS)	-500 MB
Moto / Cognito emulado	~300 MB	**0 MB** (AWS)	-300 MB
Backend Ktor (`:users:run`)	~1,200 MB	**0 MB** (Lambda)	-1,200 MB
Build APK (Gradle)	~2,500 MB	**0 MB** (de Build)	-2,500 MB
Emulador Android	~2,000 MB	~2,000 MB	0
Maestro + ADB	~400 MB	~400 MB	0
Total fase QA	**~7,400 MB**	**~2,400 MB**	**-5,000 MB (68%)**

8.2 Escenarios de RAM

Actual — QA local sin liberacion:

~22 GB / 16 GB — SWAP SEVERO

Nuevo — QA remoto + liberacion pre-QA:

~6.4 GB / 16 GB — 9.6 GB libres

Peor caso — QA remoto sin liberacion (agentes activos):

~11.5 GB / 16 GB — 4.5 GB libres

> **Resultado:** Incluso en el peor caso (sin liberar agentes), QA remoto cabe en 16 GB con margen. Con liberacion pre-QA, hay 9.6 GB libres — margen mas que suficiente para video de alta calidad.

9. Prerequisitos de Implementacion

#	Prerequisito	Estado	Accion concreta	Bloqueante
1	Agregar `workflow_dispatch` a `main.yml`	Pendiente	Agregar trigger en el workflow de GitHub Actions	Si
2	Crear datos seed en DynamoDB remoto	Pendiente	Adaptar `init-local-aws.sh` para crear negocio y usuario de test en AWS	Si
3	Crear usuario de test en Cognito remoto	Pendiente	Via consola AWS o CLI: crear `qa-test@intrale.com`	Si
4	Modificar fase Build para generar shadowJar + copiar artefactos	Pendiente	Agregar `shadowJar` al builder + copiar a `qa/artifacts/`	Si
5	Crear `qa/scripts/qa-env-up-remote.sh`	Pendiente	Script liviano: liberacion + deploy via gh + health-check	Si

6	Agregar flag `--remote` a `smoke-test.sh`	Pendiente	Flag que usa `qa-env-up-remote.sh` + APK sin LOCAL_BASE_URL	No
7	Verificar flows Maestro contra backend remoto	Pendiente	Dry-run para validar timeouts con latencia de red	No
8	Agregar warmup pre-Maestro	Pendiente	2-3 requests de calentamiento para evitar cold start de Lambda	No

10. Riesgos y Mitigaciones

Riesgo	Prob.	Impacto	Mitigacion
Latencia de red afecta tests Maestro	Media	Medio	Maestro ya espera elementos con timeout; agregar `extendedWaitUntil` si falla; warmup pre-test
Lambda cold start (primer request ~5-10s)	Alta	Bajo	3 requests de warmup antes de iniciar Maestro
Datos de test contaminados entre runs	Media	Medio	Script de limpieza/reinicio de datos al inicio de cada QA run
Deploy pisa version de produccion de Lambda	Baja	Alto	App no productiva aun; cuando lo sea, crear Lambda staging separada
Internet caido durante QA	Baja	Alto	Detectar fallo de red al inicio; fallback a modo local (legacy)
GitHub Actions sin minutos	Baja	Medio	Monitorear uso mensual; plan gratuito tiene 2000 min/mes
Workflow falla en GitHub	Baja	Medio	Reintentar 1 vez; si falla 2x, abortar QA y notificar

11. Plan de Implementacion por Fases

Fase 1 — Infraestructura (prerequisitos)

#	Tarea	Esfuerzo
1.1	Agregar `workflow_dispatch` a `.github/workflows/main.yml`	Simple
1.2	Crear datos seed en DynamoDB remoto (negocio `intrale`, usuario `qa-test@intrale.com`)	Simple
1.3	Crear/confirmar usuario de test en Cognito remoto	Simple
1.4	Verificar que `gh workflow run` funciona contra la rama actual	Simple

Fase 2 — Scripts y fase Build

#	Tarea	Esfuerzo
2.1	Modificar el skill `/builder` para generar `shadowJar` + copiar artefactos a `qa/artifacts/`	Medio
2.2	Crear `qa/scripts/qa-env-up-remote.sh` (liberacion + deploy + health-check)	Medio
2.3	Crear `qa/scripts/qa-env-down-remote.sh` (desactivar Priority Window + cleanup)	Simple
2.4	Agregar flag `--remote` a `smoke-test.sh`	Simple

Fase 3 — Integracion y validacion

#	Tarea	Esfuerzo
3.1	Modificar skill `/qa` para usar modo remoto por defecto	Medio
3.2	Dry-run completo: Build → QA remoto → evidencia	Medio
3.3	Ajustar timeouts de Maestro si latencia causa flakiness	Simple
3.4	Documentar el nuevo flujo en `docs/`	Simple

12. Resumen Ejecutivo

Problema: 16 GB de RAM es insuficiente para ejecutar QA E2E (~7.4 GB) junto con agentes de desarrollo (~5.5 GB) y el sistema operativo (~3 GB). El sistema se satura y se vuelve impredecible.

Solucion (3 medidas combinadas, costo $0):

1. **Backend remoto:** El emulador le pega al Lambda/API Gateway que ya existe en AWS. Elimina Docker + DynamoDB local + Moto + Backend Ktor = **-2.5 GB**
2. **APK de fase Build:** QA no recompila — usa el APK que la fase Build ya genero = **-2.5 GB**
3. **Liberacion pre-QA:** Activar Priority Window + matar residuales antes de arrancar = **+3-5 GB libres**

Resultado: QA pasa de ~7.4 GB a ~2.4 GB. Reduccion del 68%.

Deploy del backend: Via `gh workflow run` (GitHub Actions). El workflow existente compila el JAR con secrets inyectados y lo deploya a Lambda. Cero RAM local, cero secrets en la maquina. Solo requiere agregar `workflow_dispatch` al trigger del workflow.

Emulador: Se mantiene en 1536 MB / 2 cores para garantizar calidad de video y estabilidad de tests.

Paralelismo: QA sigue siendo secuencial (1 issue a la vez). No cambia respecto a hoy. Para el futuro, crear Lambda staging separada cuando la app sea productiva.

Prerequisito critico: Agregar `workflow_dispatch` a `main.yml` + crear datos de test en AWS remoto.

Reporte consolidado — Intrale Platform — Saturacion QA: Analisis, Estrategias y Plan de Implementacion
2026-04-06 | Claude Code (Opus 4.6)